

Data Visualization Using ggplot2

Dr. Mutua Kilai, PhD

September-December 2026

Department of Pure and Applied Sciences

Introduction

- We deal with data visualization in this lecture.

Packages

- We will make use of the following packages:
 - i. `ggplot2`
 - ii. `patchwork`

Getting started

Quote

The simple graph has brought more information to the data analyst's mind than any other device.” — John Tukey

- All the plots are composed of the **data**, the information you want to visualize and a **mapping** the description of how the data variables are mapped to aesthetic attributes.

Mapping components

Layer

A layer is a collection of geometric elements and statistical transformations. Geometric elements, geoms for short, represent what you actually see in the plot: points, lines, polygons, etc. Statistical transformations, stats for short, summarise the data: for example, binning and counting observations to create a histogram, or fitting a linear model.

Scale

Scales map values in the data space to values in the aesthetic space. This includes the use of colour, shape or size. Scales also draw the legend and axes, which make it possible to read the original data values from the plot (an inverse mapping).

Mapping components

coord

A coord, or coordinate system, describes how data coordinates are mapped to the plane of the graphic. It also provides axes and gridlines to help read the graph. We normally use the Cartesian coordinate system, but a number of others are available, including polar coordinates and map projections.

facet

A facet specifies how to break up and display subsets of data as small multiples. This is also known as conditioning or latticing/trellising.

theme

A theme controls the finer points of display, like the font size and background colour. While the defaults in ggplot2 have been chosen with care, you may need to consult other references to create an attractive plot.

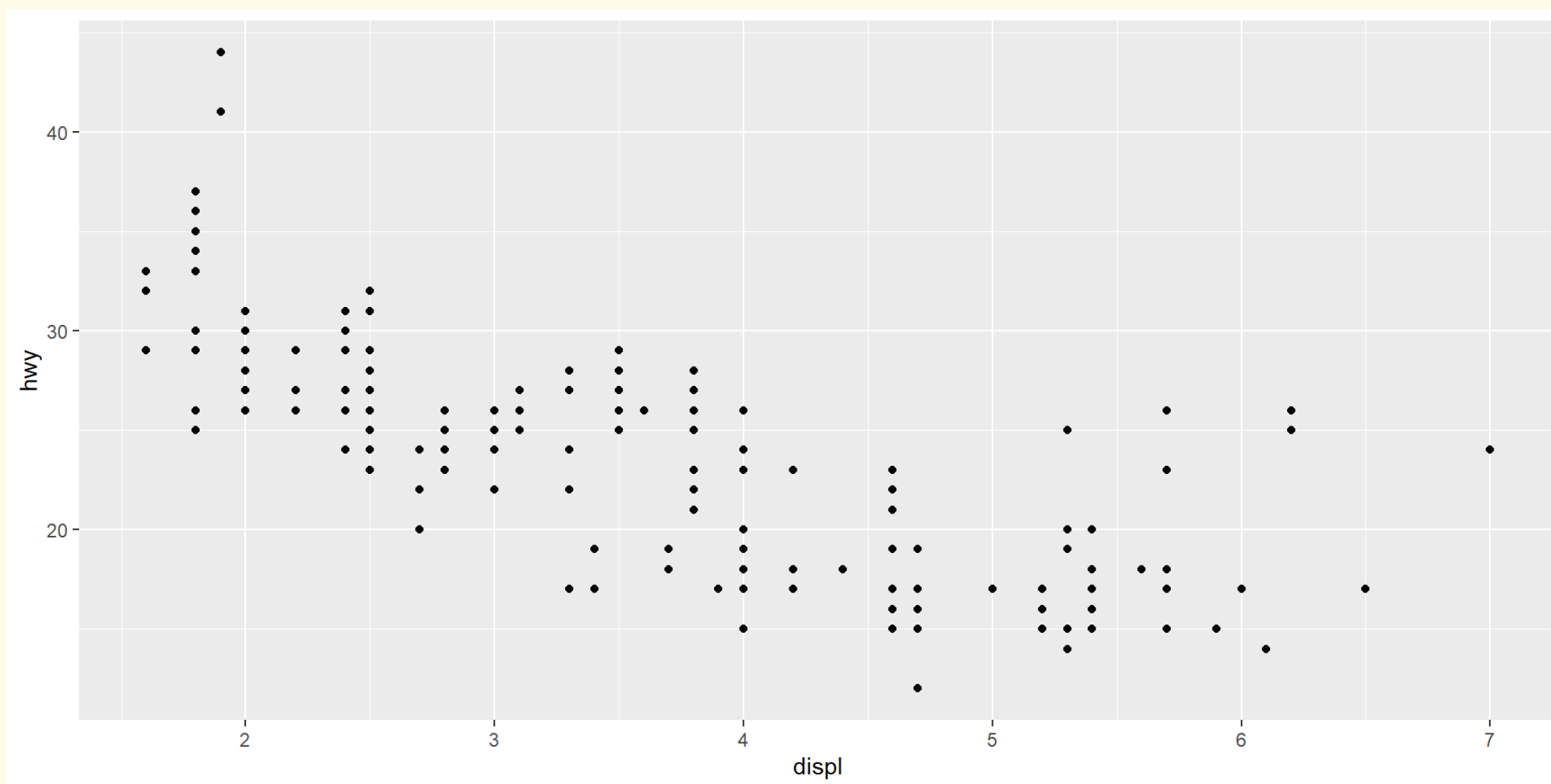
Key component of `ggplot`

- Every `ggplot2` plot has three key components:
 - i. **data**
 - ii. A set of **aesthetic mapping** between variables in the data and visual properties
 - iii. At least one layer which describes how to render each observation. Layers are usually created with a **geom** function

Scatter plot

- Explores the relationship between two variables

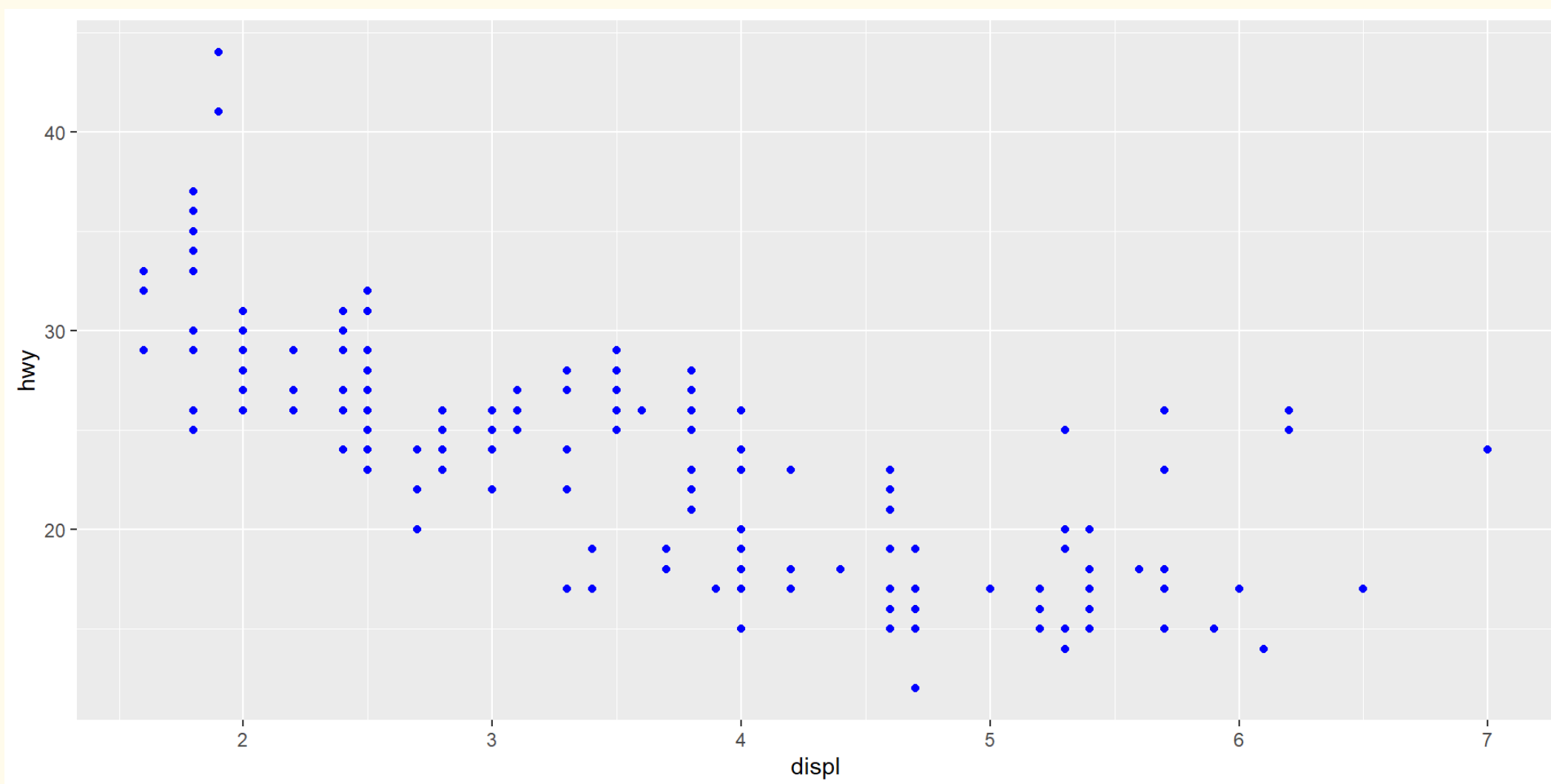
```
1 library(ggplot2)
2 ggplot(data = mpg, aes(x = displ, y = hwy)) +
3   geom_point()
```



Color the points with a uniform color

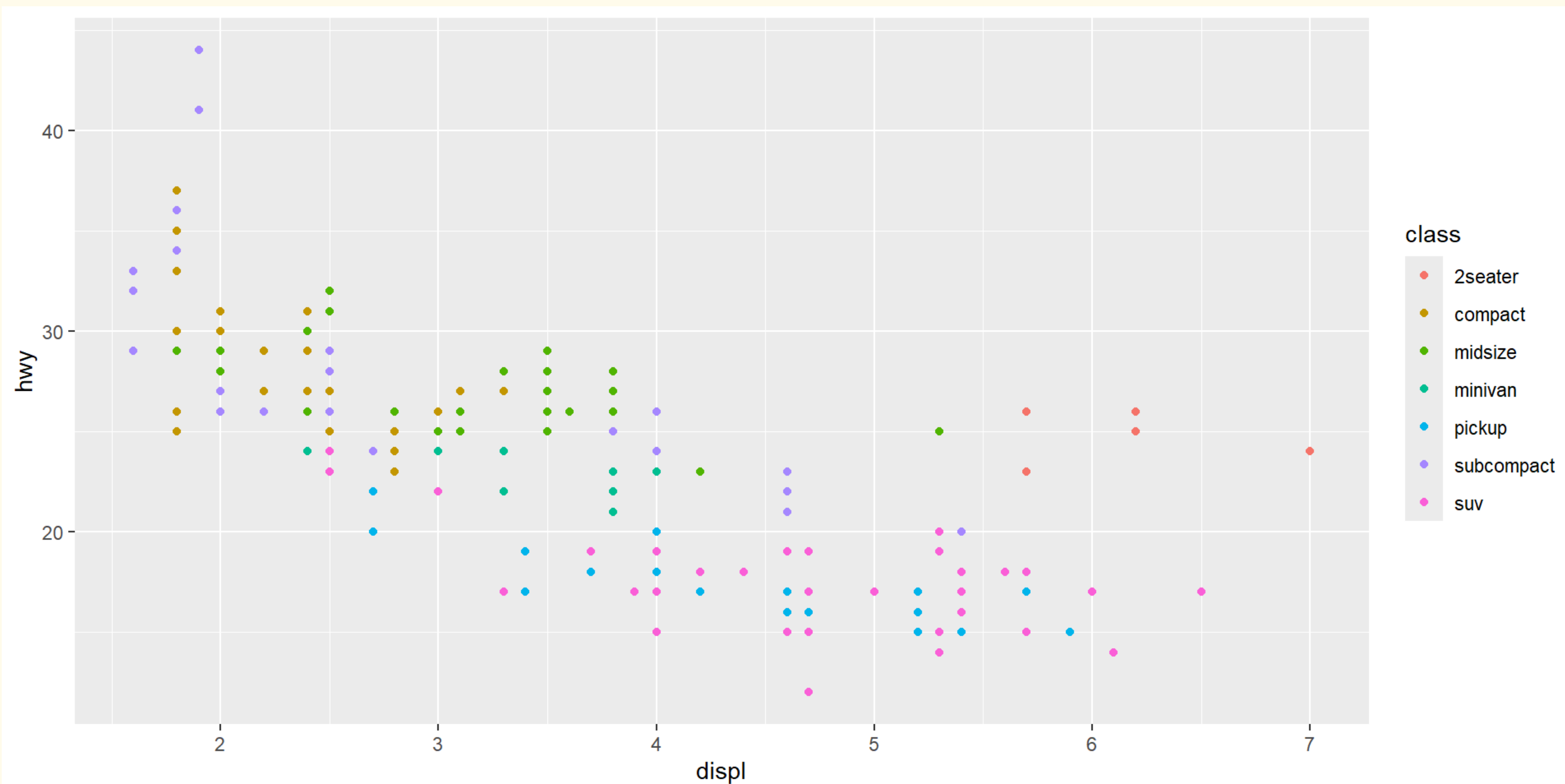
- We can color the points in the scatter plot with any color of choice by adding `colour` attribute

```
1 ggplot(mpg, aes(displ, hwy)) + geom_point(colour = "blue")
```



Colour by a different variable

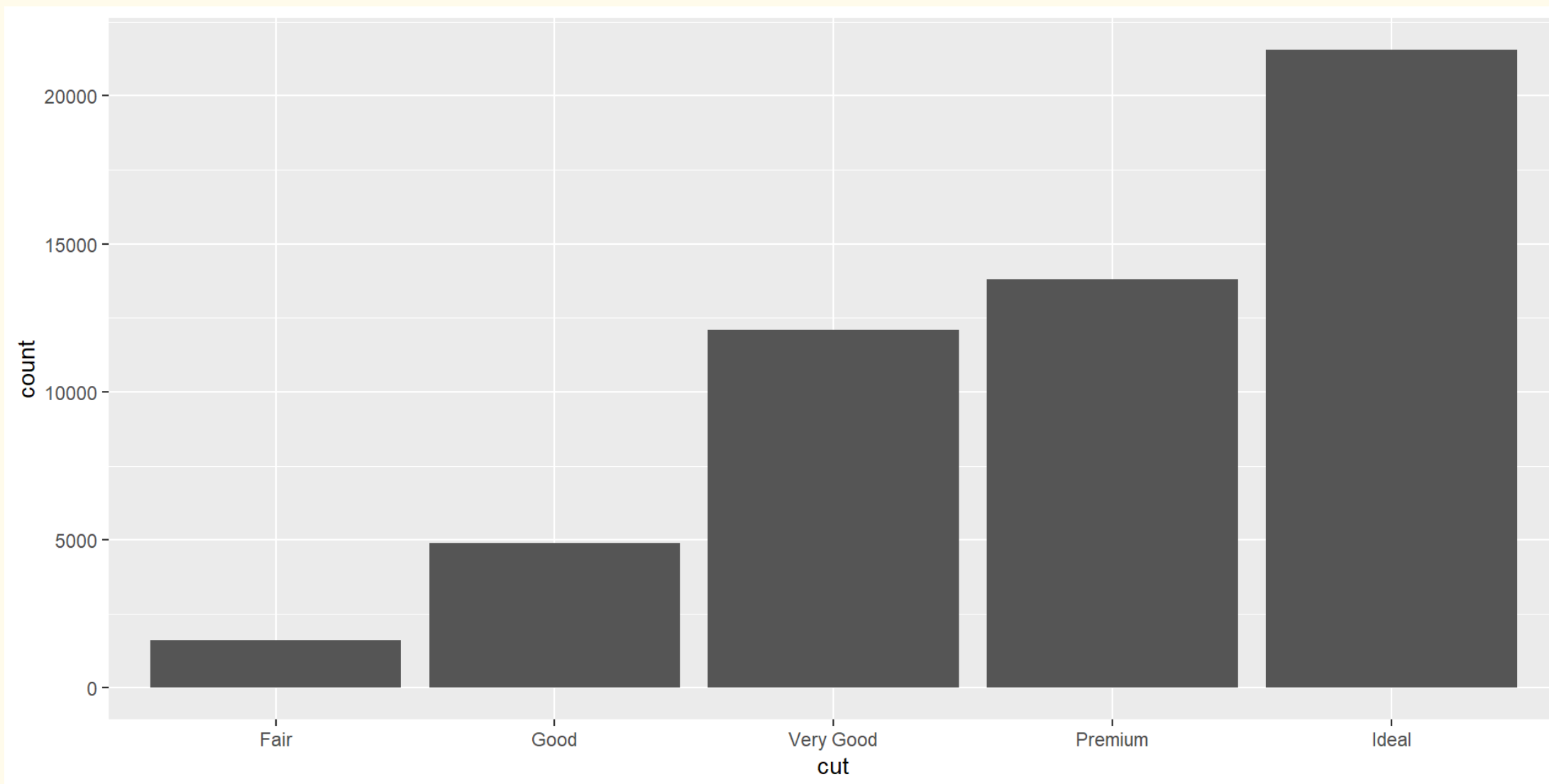
```
1 ggplot(mpg, aes(displ, hwy, colour = class)) +  
2   geom_point()
```



Simple Bar graph

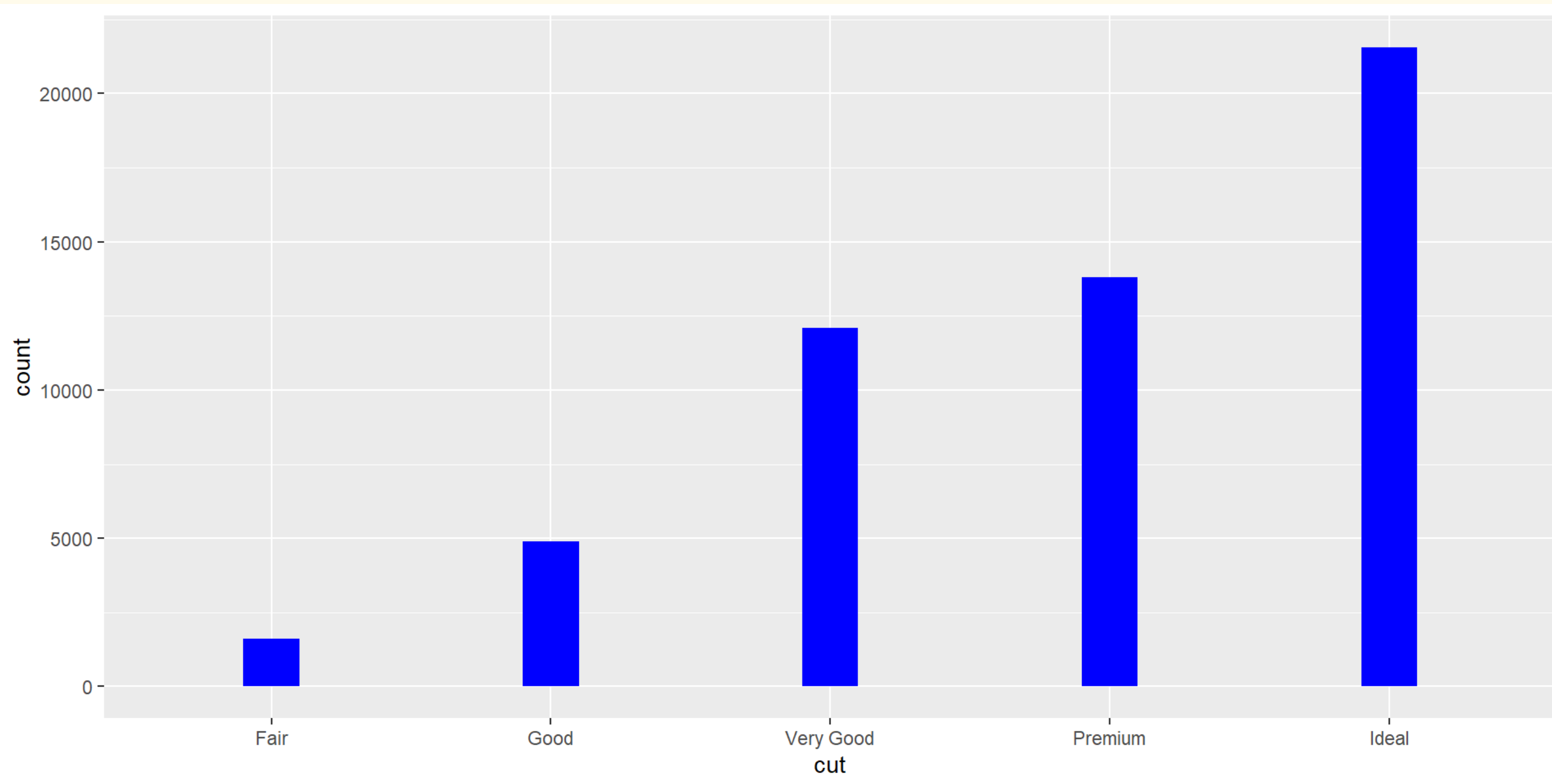
- Used to visualize a categorical variable

```
1 ggplot(data = diamonds) +  
2   geom_bar(mapping = aes(x = cut))
```



Change color and width of bars

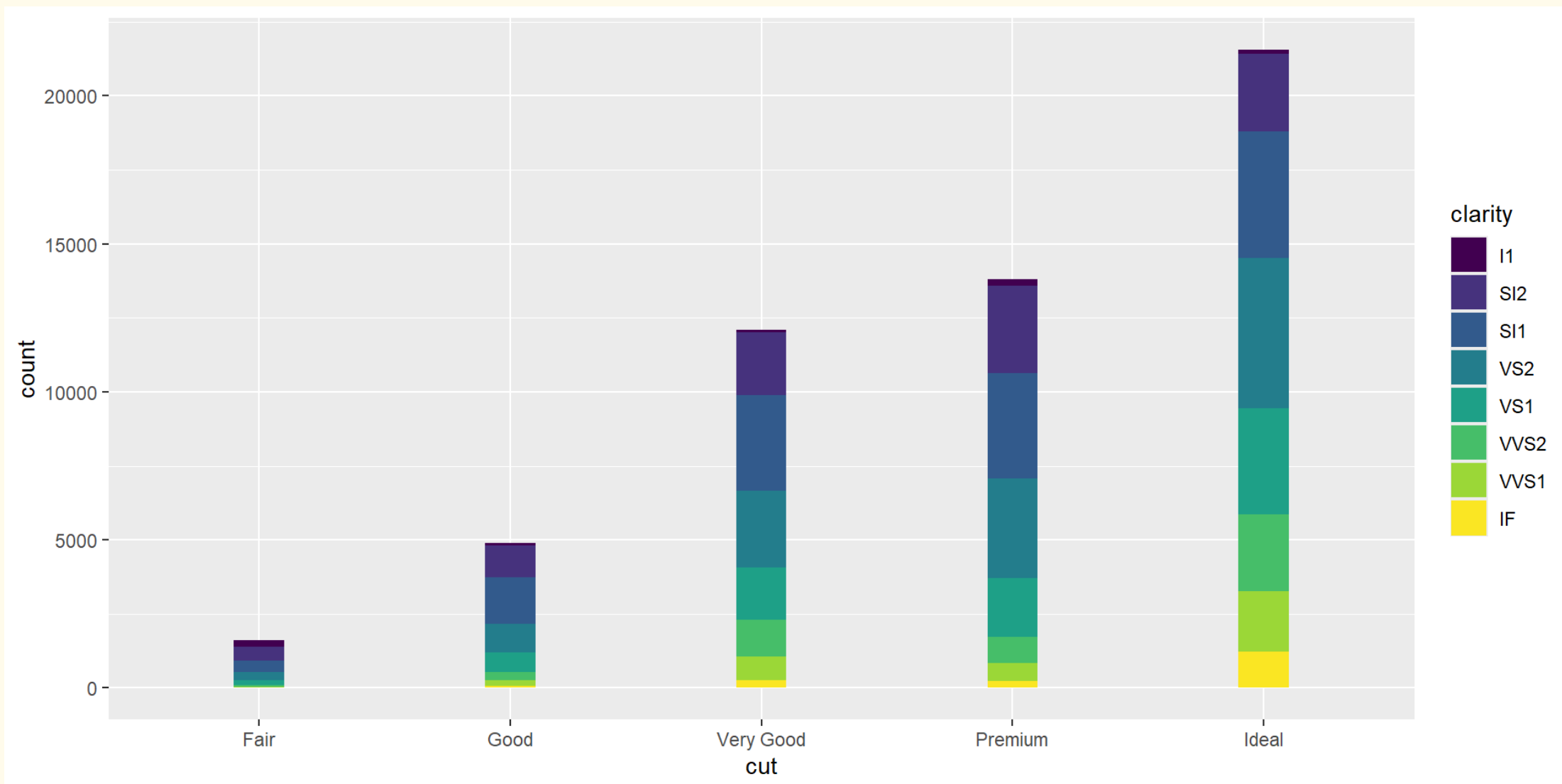
```
1 ggplot(data = diamonds) +  
2   geom_bar(mapping = aes(x = cut), width = 0.2, fill="blue")
```



Stacked bar graph

- This helps visualize two categorical variables

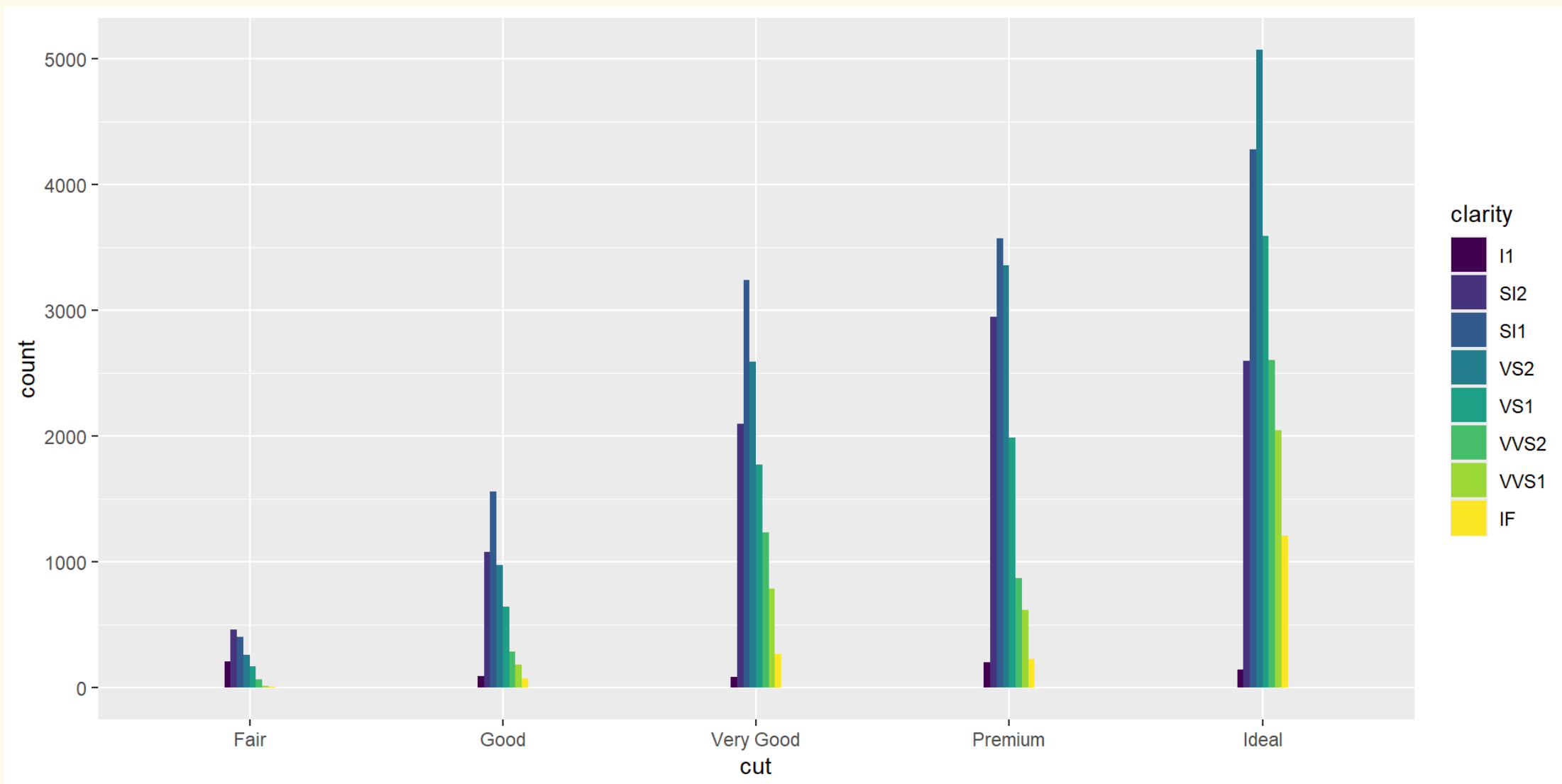
```
1 ggplot(data = diamonds) +  
2   geom_bar(mapping = aes(x = cut, fill = clarity), width = 0.2)
```



Clustered bar graph

- This helps visualize two categorical variables

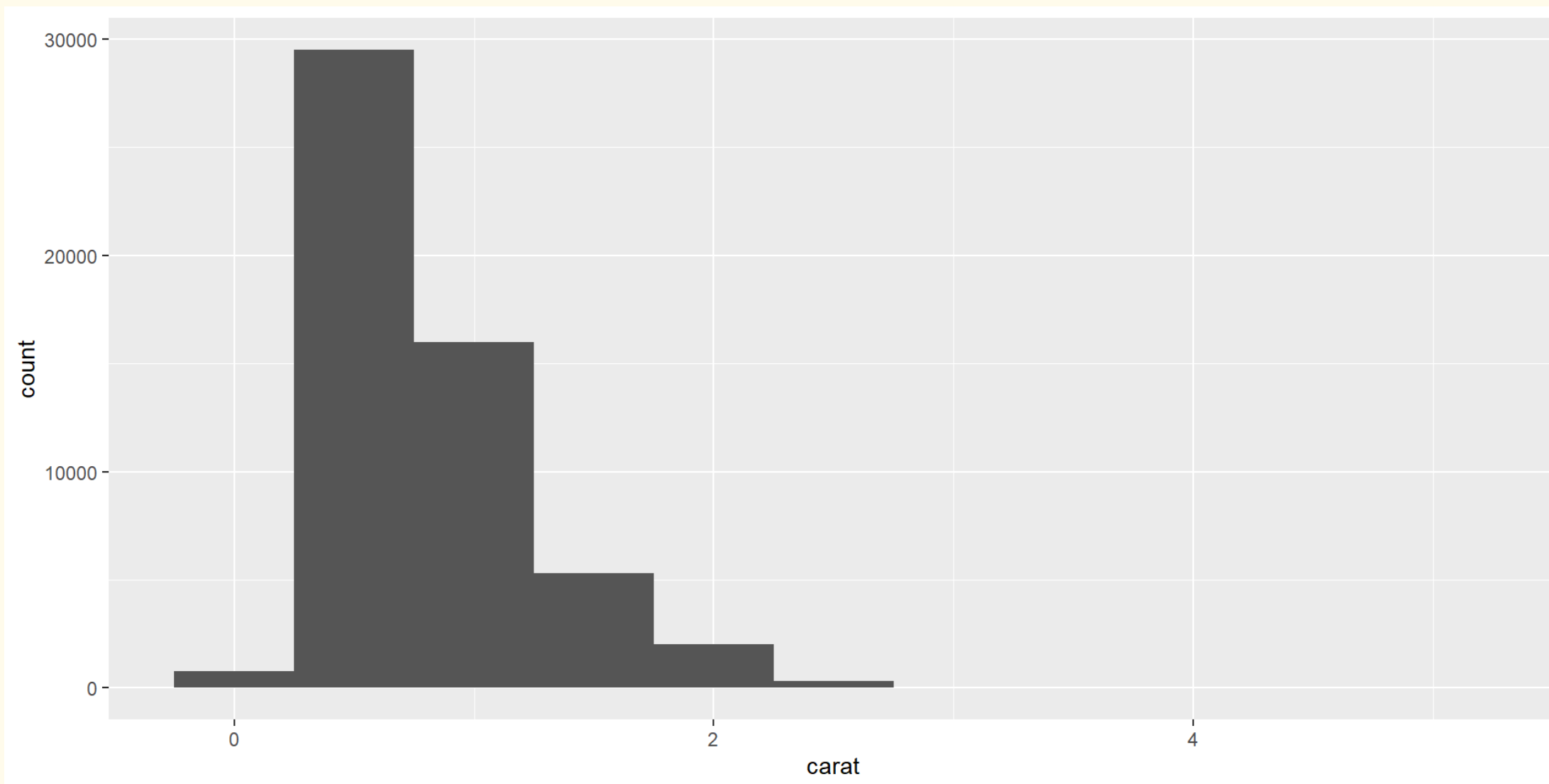
```
1 ggplot(data = diamonds) +  
2   geom_bar(mapping = aes(x = cut, fill = clarity), width = 0.2, position = "dodge")
```



Histogram

- To visualize the distribution of continuous variable we use histogram

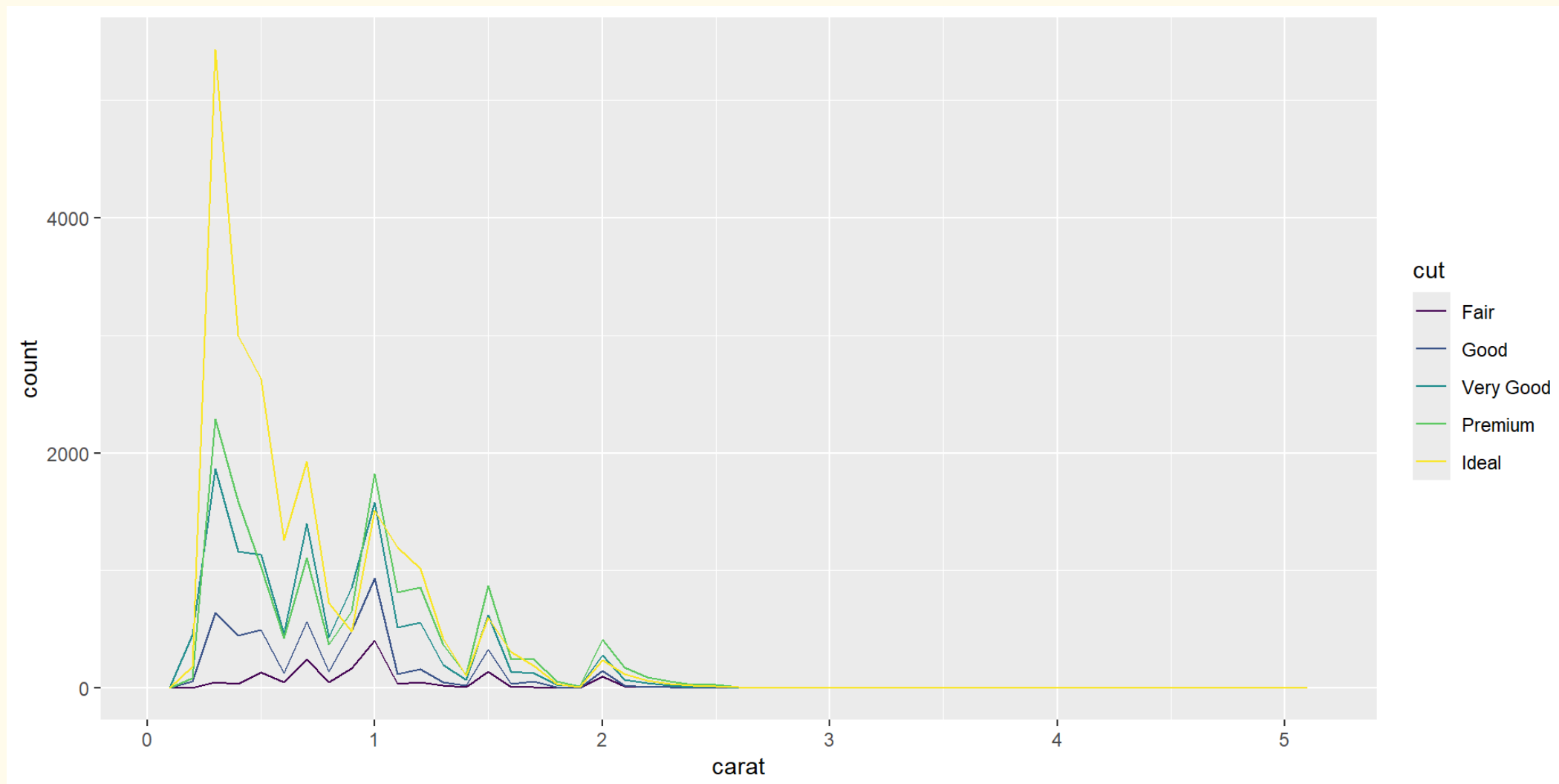
```
1 ggplot(data = diamonds) +  
2   geom_histogram(mapping = aes(x = carat), binwidth = 0.5)
```



Frequency Polygon

- If you wish to overlay multiple histograms in the same plot, recommend using `geom_freqpoly()` instead of `geom_histogram()`

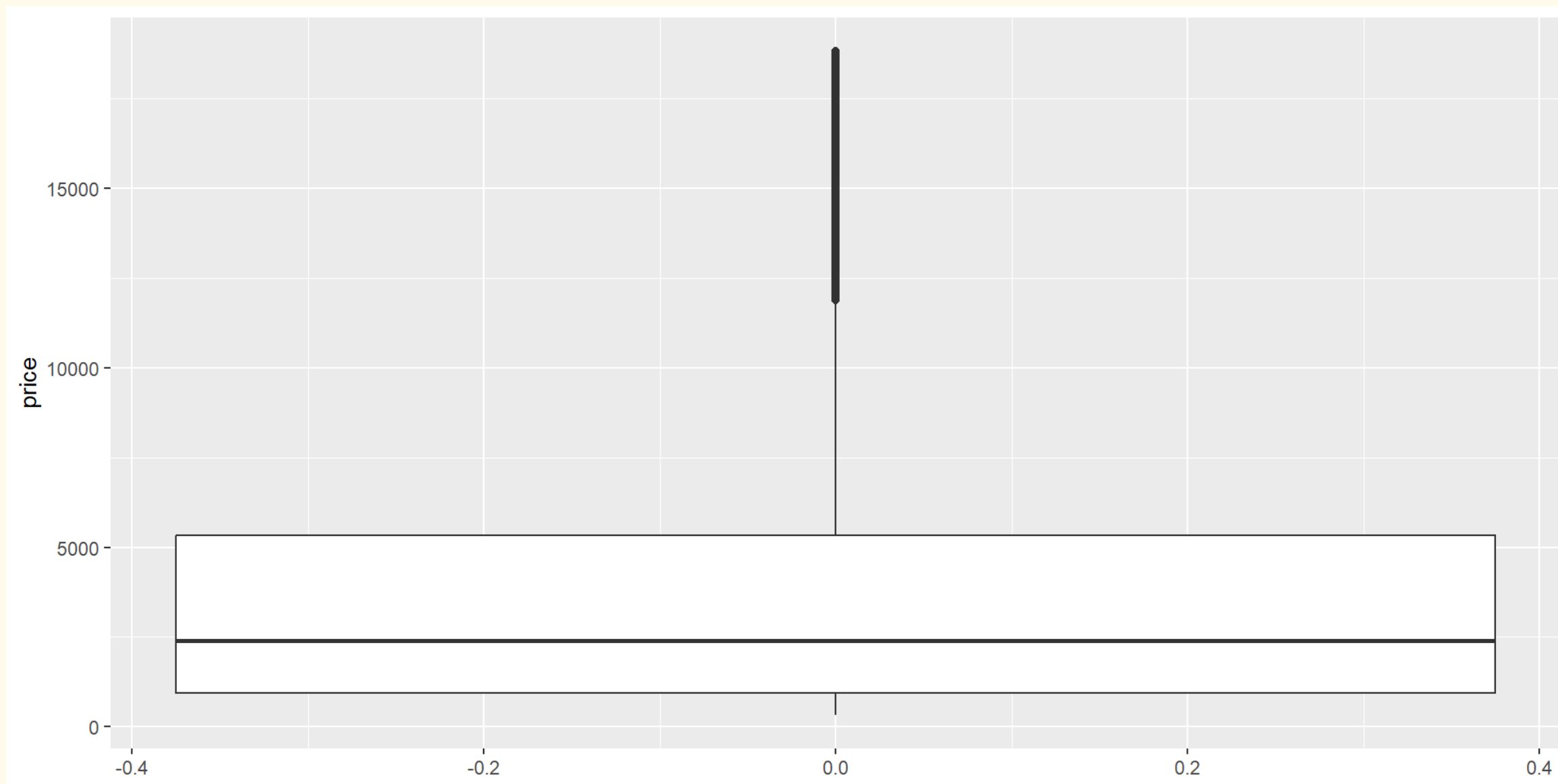
```
1 ggplot(data = diamonds, mapping = aes(x = carat, colour = cut)) + geom_freqpoly(binwidth = 0.1)
```



Boxplot - one continuous variable

- Boxplot gives numerical summaries of a continuous variable

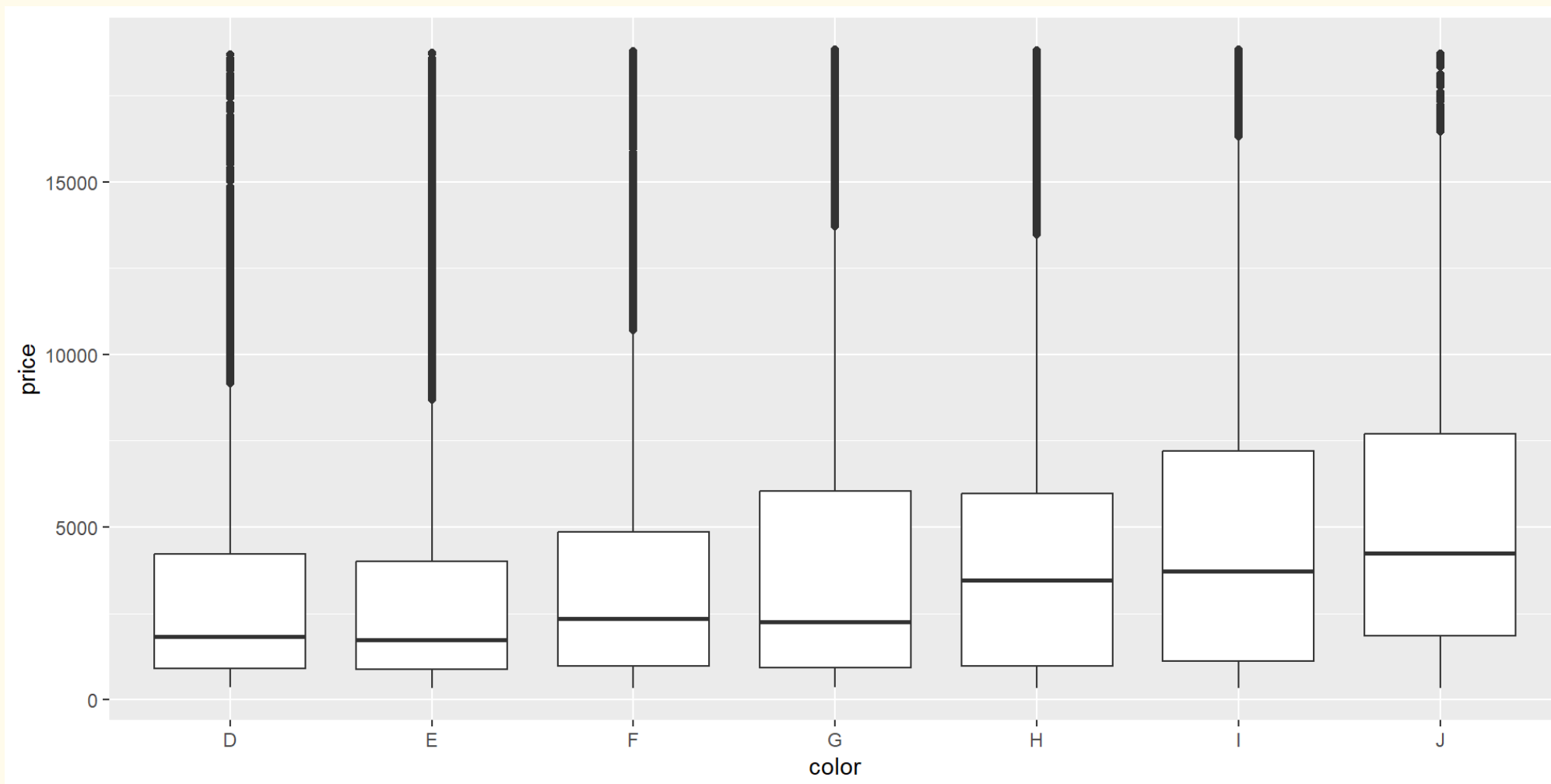
```
1 ggplot(data = diamonds, mapping = aes(y = price)) + geom_boxplot()
```



Boxplot - one continuous variable vs categorical variable

- Boxplot gives numerical summaries of a continuous variable per each category

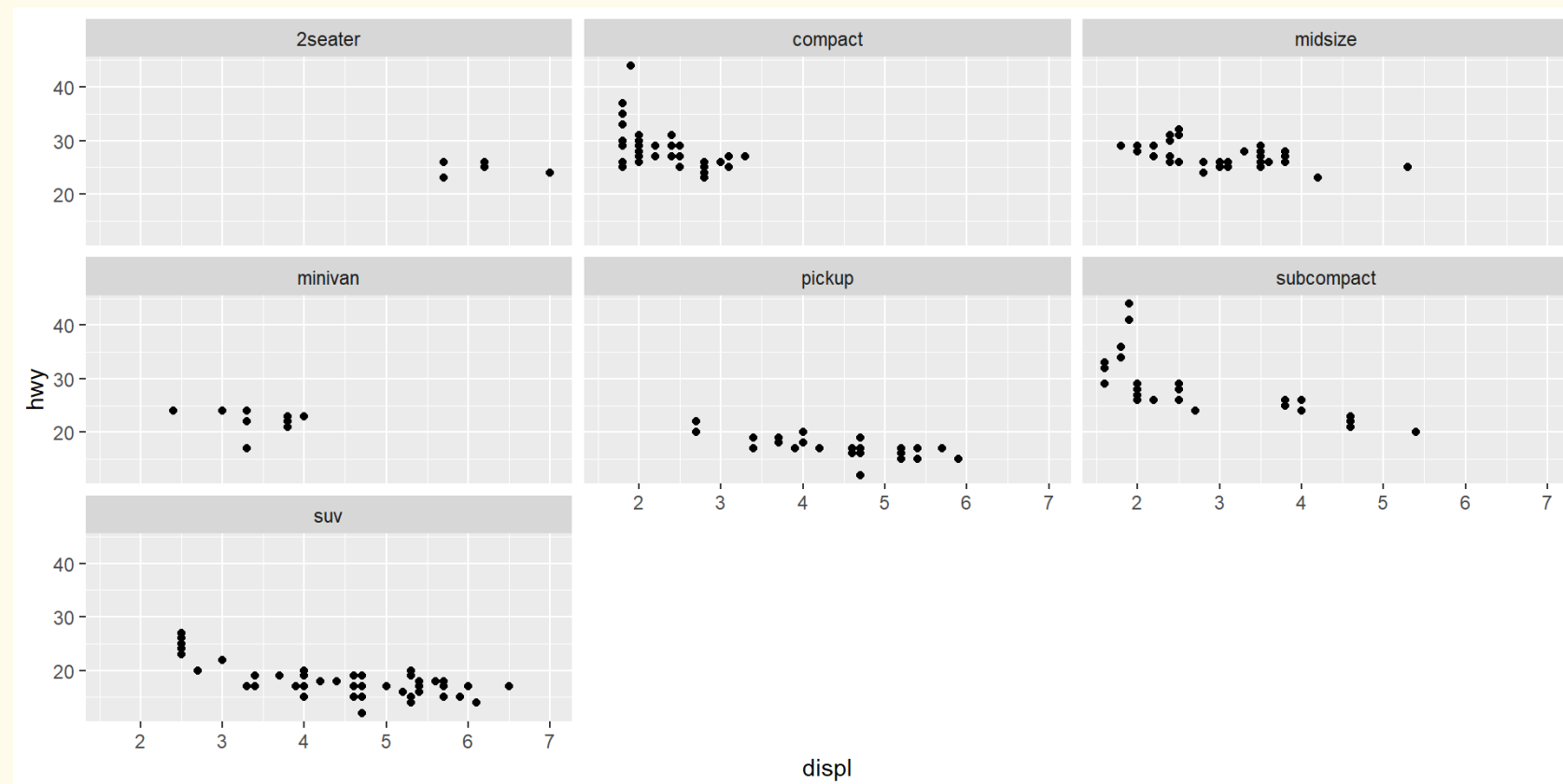
```
1 ggplot(data = diamonds, mapping = aes(x= color,y = price)) + geom_boxplot()
```



Faceting

- Another technique for displaying additional categorical variables on a plot is faceting. Faceting creates tables of graphics by splitting the data into subsets and displaying the same graph for each subset.

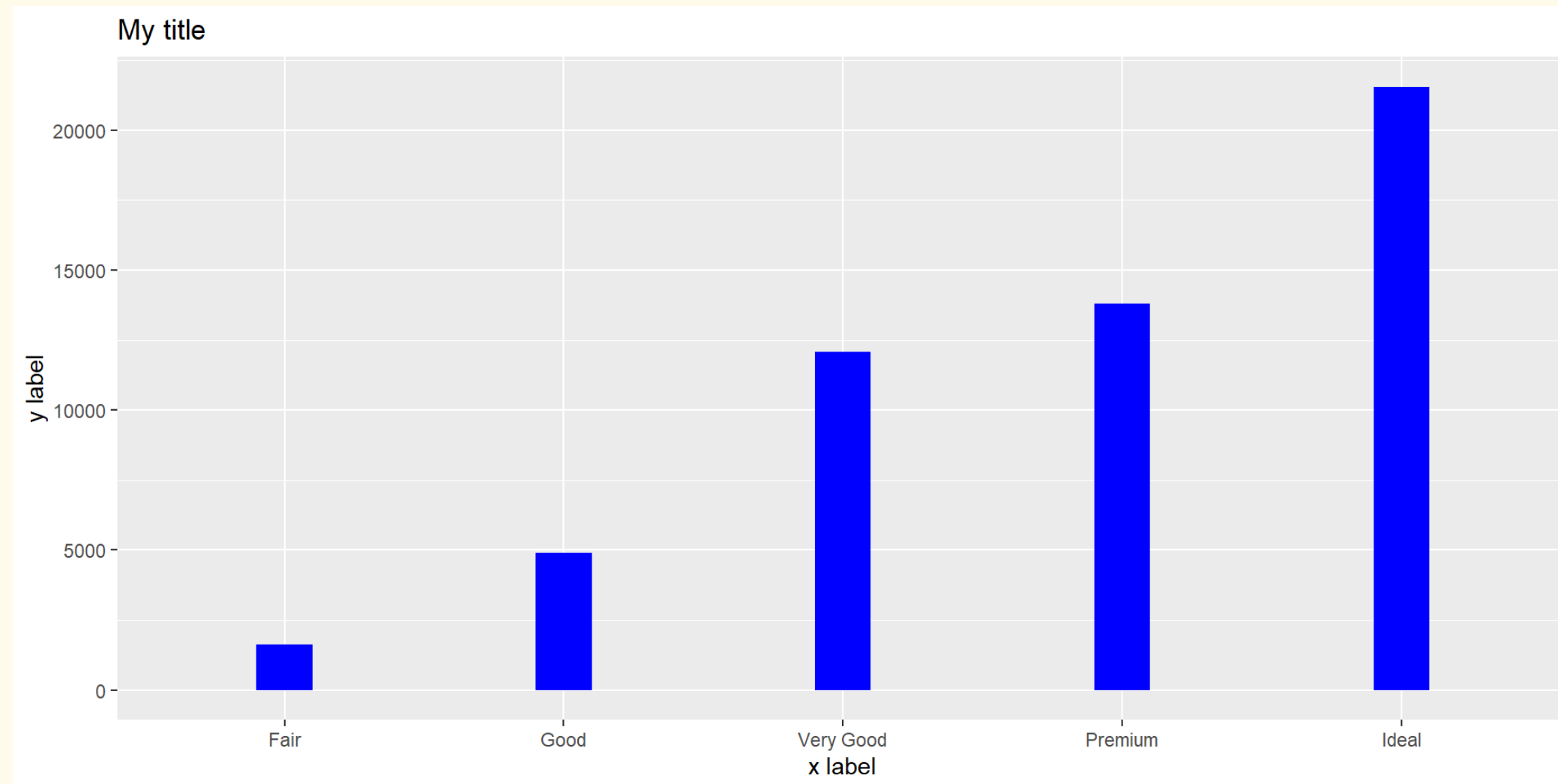
```
1 ggplot(mpg, aes(displ, hwy)) +  
2   geom_point() +  
3   facet_wrap(~class)
```



Adding titles and labels to axis

- We can add titles and label to axis by adding the `labs` in `ggplot2`

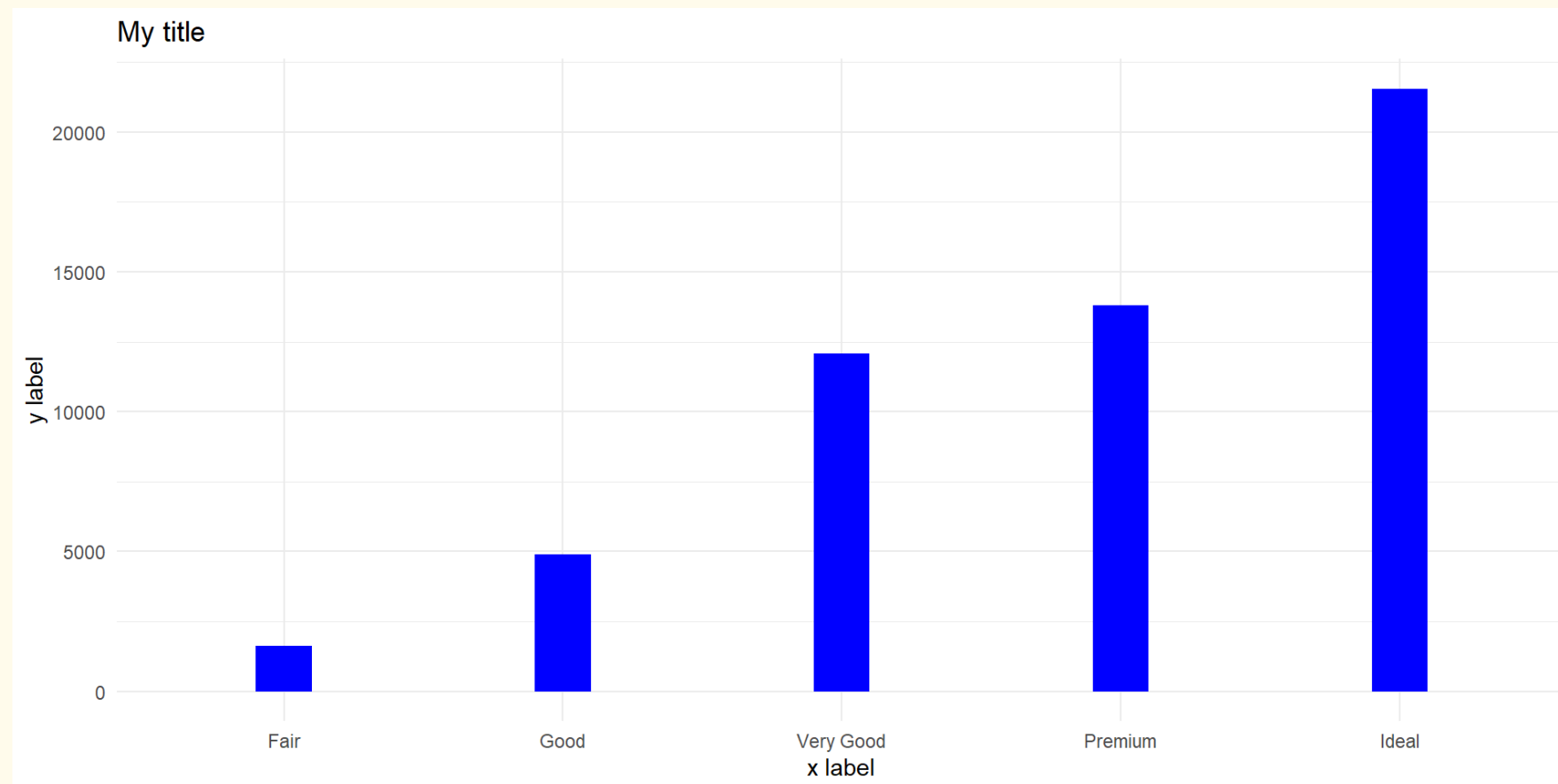
```
1 ggplot(data = diamonds) +  
2   geom_bar(mapping = aes(x = cut), width = 0.2, fill="blue")+  
3   labs(title = "My title", x = "x label", y = "y label")
```



Themes in ggplot2

- Themes help you customize the appearance of your plots. You can add a theme by adding `theme_` to the plots

```
1 ggplot(data = diamonds) +  
2   geom_bar(mapping = aes(x = cut), width = 0.2, fill="blue")+  
3   labs(title = "My title", x = "x label", y = "y label")+  
4   theme_minimal()
```



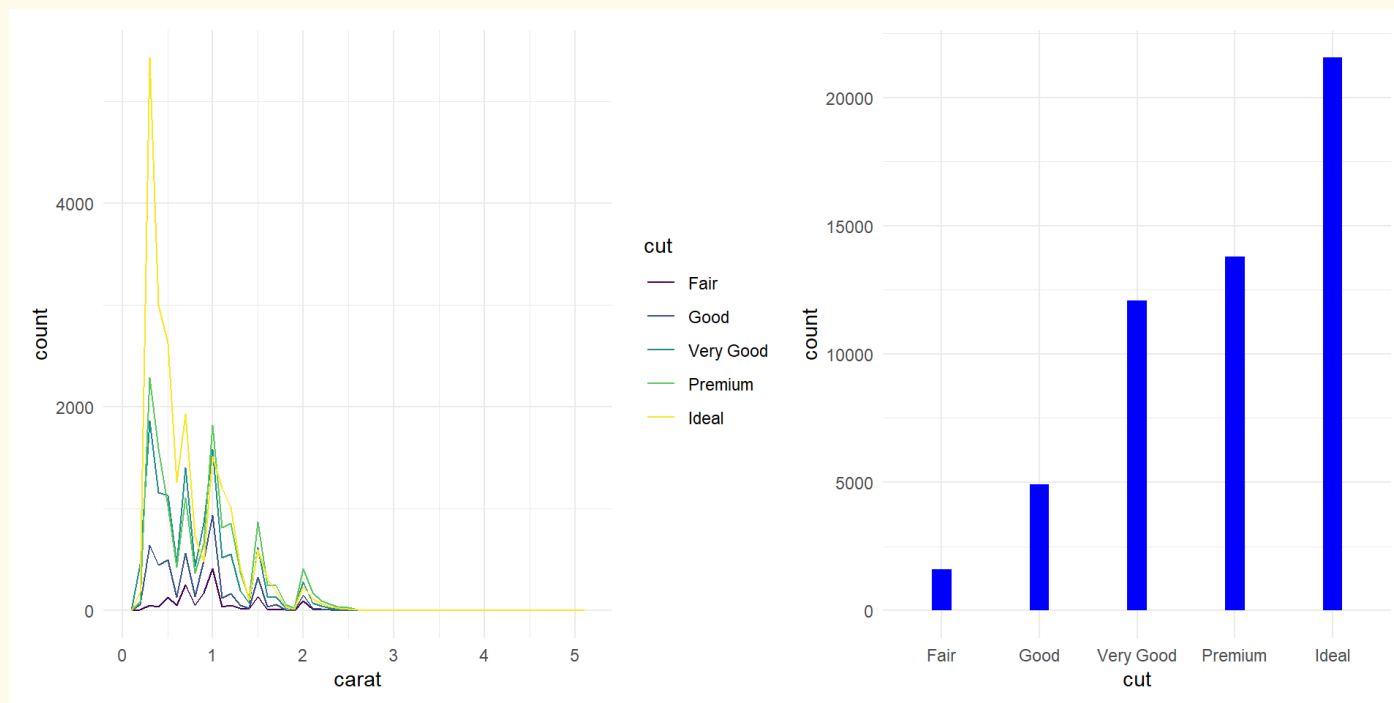
Combining plots using patchwork

- One can be able to combine multiple plots using `patchwork` as follows

```

1 library(patchwork)
2
3 p1 <- ggplot(data = diamonds, mapping = aes(x = carat, colour = cut)) + geom_freqpoly(binwidth = 0.1) + theme_minimal()
4
5 p2 <- ggplot(data = diamonds) +
6   geom_bar(mapping = aes(x = cut), width = 0.2, fill="blue")+
7   theme_minimal()
8
9 p1 + p2

```

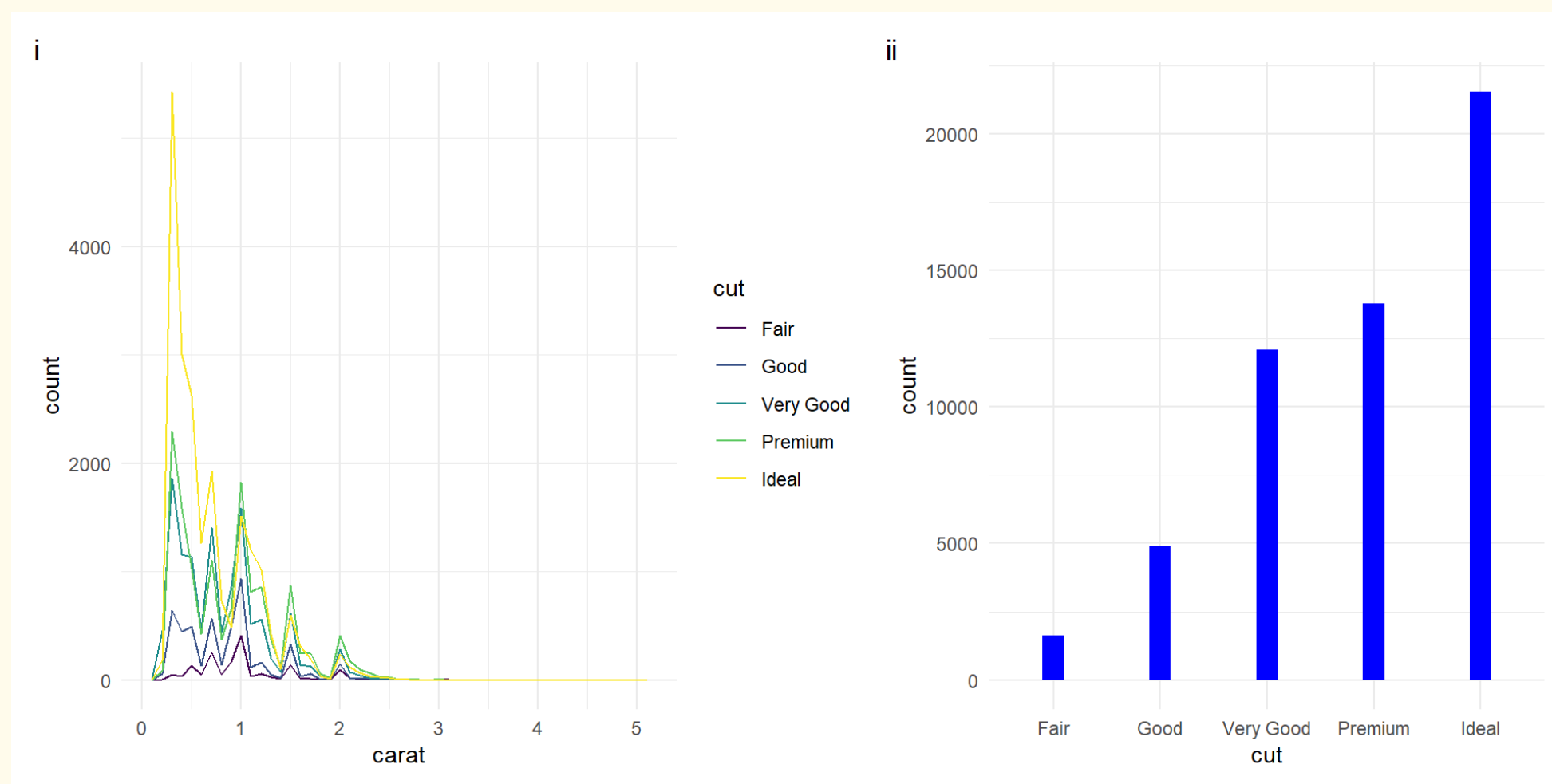


Annotating using patchwork

```

1 library(patchwork)
2
3 p1 <- ggplot(data = diamonds, mapping = aes(x = carat, colour = cut)) + geom_freqpoly(binwidth = 0.1) + theme_minimal()
4
5 p2 <- ggplot(data = diamonds) +
6   geom_bar(mapping = aes(x = cut), width = 0.2, fill="blue")+
7   theme_minimal()
8
9 p1 + p2 + plot_annotation(tag_levels = "i")

```



Thank You